# BMW Group Develops a GenAI Assistant to Accelerate Infrastructure Optimization on AWS

by Dr. Jens Kohl, Alexander Wöhlke, Brendan Jones, Christian Mueller, Daniel Engelhardt, Liza (Elizaveta) Zinovyeva, Nuno Castro, Satyam Saxena, and Shukhrat Khodjaev | on 15 MAR 2024 | in Amazon Bedrock, Automotive, Generative AI, Industries | Permalink | Comments | Share

BMW Group and Amazon Web Services (AWS) collaborated on a groundbreaking In-Console Cloud Assistant (ICCA) solution designed to empower hundreds of BMW DevOps teams to streamline their infrastructure optimization efforts. The ICCA solution harnesses the power of generative artificial intelligence (GenAI) on AWS to provide real-time insights and recommendations that help accelerate the identification and resolution of performance bottlenecks, resource optimization opportunities, cost-saving measures and more. The ICCA helps to address the challenge of navigating the intricacies of cloud infrastructure optimization, which can be a daunting task for organizations managing a vast array of AWS accounts that are used to host business critical applications.

GenAI is rapidly transforming businesses and unlocking new possibilities across industries. As one of the global leaders in the automotive space, BMW has been at the forefront of using AI and machine learning (ML) to enhance customer experiences and help drive innovation. After years of experience using AI for applications such as predictive maintenance and autonomous driving, BMW is now using large language models (LLMs) in Amazon Bedrock to implement ICCA on AWS.

By 2023, BMW Group's Connected Vehicle department had amassed a robust cloud infrastructure, comprising over 1,300 microservice applications, supported by more than 450 DevOps teams utilizing more than 450 AWS accounts on a daily basis. Recognizing the need for a scalable optimization solution, BMW collaborated with the AWS Generative AI Innovation Center, an AWS program that fosters collaboration between companies and AWS experts, and AWS Professional Services. This collaboration yielded the ICCA, a conversational generative AI solution built on AWS that helps empower BMW teams across four use cases:

1. Searching and retrieving AWS services related information through questions & answers;

2. Monitoring health and detecting infrastructure issues or optimization opportunities in onboarded AWS accounts;

3. Generating code snippets; and

4. Deploying the code changes to optimize the infrastructure in particular AWS accounts [as chosen by BMW].

Built using Amazon Bedrock, the ICCA reviews BMW's workloads against pillars of the AWS Well-Architected Framework and BMW best practices, a reference framework which helps organizations build secure, reliable, efficient, cost-effective, and sustainable workloads in the AWS cloud.

The ICCA solution understands natural language requests and accesses insights from two sources: AWS Trusted Advisor, which helps users optimize costs, improve performance, and address security gaps, and AWS Config, which assesses, audits, and evaluates resource configurations (see figure 1). Additionally, the ICCA has access to always up-to-date AWS Documentation and BMW best practices.

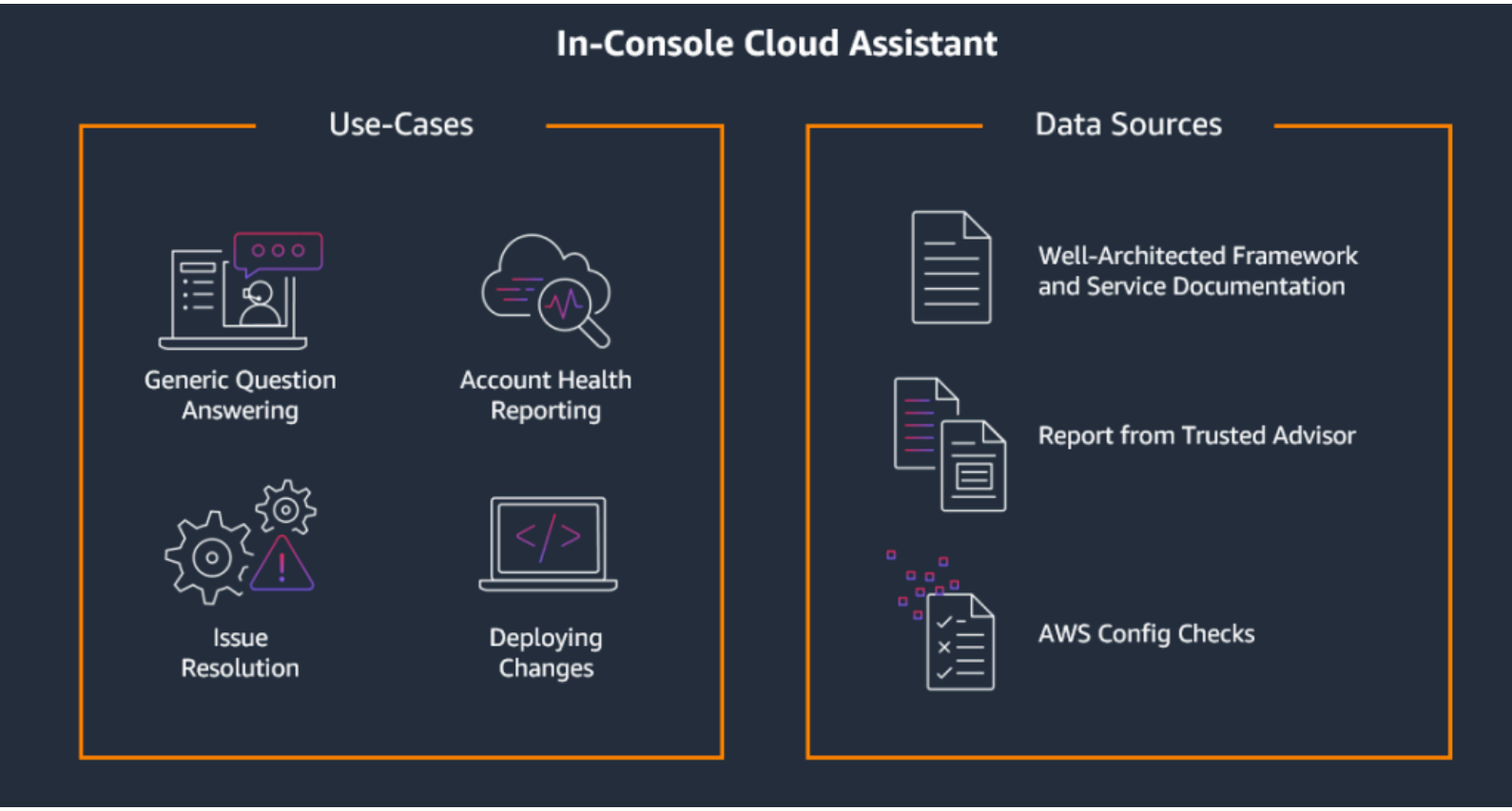


*Figure 1. In-Console Cloud Assistant – overview*

This project between BMW and AWS highlights the transformative impact of LLMs. With the ability to encode human expertise and communicate in natural language, generative AI can help augment human capabilities and allow organizations to harness knowledge at scale. The ICCA is just one example of how BMW is using AWS's industry-leading AI services to help drive innovation.

In this blog post, we provide a detailed overview of the ICCA solution by diving deep into each use-case and the underlying workflows.

## Solution Overview

In this section, we present an overview of the ICCA and its architecture. As shown in the solution architecture diagram in figure 2 below, the fully managed assistant service provides a chat-bot experience via a user interface (UI) allowing BMW DevOps teams to interact with the LLMs (1, 2, 3). It understands natural language requests and uses Amazon Bedrock to derive user intent (4). Once the intent is determined, the solution routes the request to the appropriate use-cases including general chat on AWS service-related technical topics (6), account-level view into infrastructure health using AWS Trusted Advisor and AWS Config (7), recommendations for detected issue fixes and further infrastructure optimization opportunities (8), and implementation of infrastructure improvements in respective AWS accounts (9, 10). The conversational memory is stored using Amazon DynamoDB (5) so the agents can share conversations. To answer AWS service-related questions and to recommend infrastructure optimization actions, the solution uses Amazon Kendra to index AWS documentation and create secure, generative AI-powered conversational experiences.

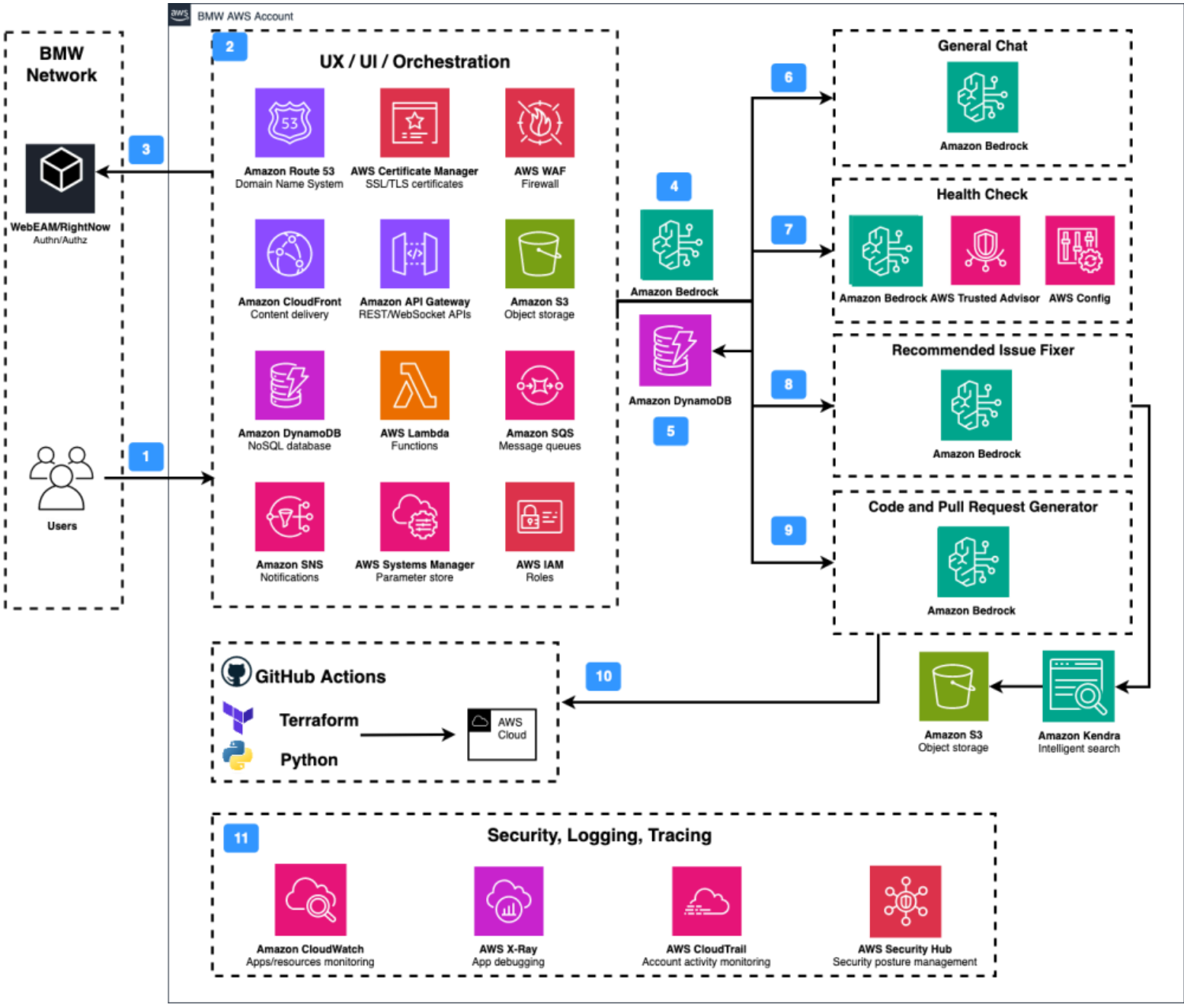


*Figure 2. In-Console Cloud Assistant architecture*

The core of the ICCA is the multi-agent system (4-10). Each LLM agent is an intelligent system designed to reason, make decisions, and take actions using the LLM and available tools — the interfaces to services, functions, and APIs, such as abilities to use search mechanisms or execute the code.

Implementing a multi-agent system has a range of advantages. First, it fosters modular development, debugging, and testing of the system. Secondly, multi-agent design enables responsibility separation between different components or functions of the system. This makes the agents more controllable and secure as each agent's behavior, inputs and outputs, can be separately monitored, tested, and equipped with security guardrails.

In the next section, we describe the main ICCA components and steps of setting up the multi-agent system.

## Step 1: Building the retrieval augmented pipeline with Amazon Kendra

The ICCA uses a retrieval augmented pipeline to find relevant content from AWS documentation and BMW best practices in order to generate responses. Additionally, it ensures constantly up-to-date documentation without need for costly retraining of the model. The pipeline has the following components:

### Ingesting documents in Amazon Kendra

The first step ingests technical AWS documentation into Amazon Kendra to create a searchable index. We use the Amazon

Kendra console to create an index and configure data sources, which is shown in figure 3 below. The assistant utilizes the following data sources: (1) AWS Well-Architected Framework whitepapers and (2) relevant documentation for services such as Amazon EC2, Amazon RDS, Amazon S3, AWS Lambda, etc. and is easily extendable to other documents.

*Figure 3. Kendra console*

*Figure 4. S3 Overview of AWS Documentation used for ICCA's Kendra index*

We upload these documents to Amazon Simple Storage Service (Amazon S3), an object storage service built to retrieve any amount of data from anywhere, as shown in figure 4. Then, within the Amazon Kendra console, we create crawler jobs to index the documents.

**Retrieval Augmented Generation (RAG)**

The relevant excerpts from Amazon Kendra are joined and used to construct a prompt for the LLMs. The LLM uses this prompt to generate a response for the user query.

Calling the Amazon Kendra retriever API:

```
import boto3
client = boto3.client("kendra")
query = "Machine Learning"
response = client.retrieve(IndexId=<Index_id>, QueryText=query)
```

Response from the Amazon Kendra retriever API:

```
[{
   "QueryId":"abcd",
   "ResultItems":[
      {
         "Id":"abcd",
         "DocumentId":"s3://...",
         "DocumentTitle":"machine-learning-foundations.pdf",
         "Content":"Amazon Web Services Machine Learning Foundations Page 6 Problems that were intr
         "DocumentURI":"https://kendra-awsdocs-bmw-poc-bucket.s3.amazonaws.com/AWS_Whitepapers/Mach
         "DocumentAttributes":[
            {
               "Key":"_source_uri",
               "Value":{
                  "StringValue":"https://kendra-awsdocs-bmw-poc-bucket.s3.amazonaws.com/AWS_Whitepa
               }
            },
            {
               "Key":"s3_document_id",
```

Calling the Amazon Bedrock model:

```
from langchain.llms.bedrock import Bedrock

bedrock_model_endpoint = Bedrock(region_name=<REGION_NAME>,
                                 model_id=<MODEL_ID>,
                                 model_kwargs=<MODEL_PARAMETERS>)

prompt = f"""Human:
            Summarize the following search results :
            {response['ResultItems'][0]['Content']}
            Assistant:
        """
```

Response from the Amazon Bedrock model:

> Here is a summary of the search results:
>
> – Machine learning has enabled significant progress in AI, solving complex problems like vision, natural language, speech, motion planning etc. with human-level accuracy.
>
> – Tasks like answering phone calls or playing chess at a master level are becoming increasingly difficult for machines to handle. However, machine learning has enabled machines to function in more human-like ways, including having directed conversations.
>
> – Machine learning has become an established branch of computer science and is key to solving practical AI problems.
>
> – Modern AI relies primarily on machine learning, not symbolic knowledge representations. Machine learning, including neural networks and deep learning, form the foundation for most AI applications today across various domains.

The RAG approach helps makes sure the LLM generates responses are grounded in factual AWS documentation, which avoids hallucinations. The relevant matches from Amazon Kendra guide the LLM so that it focuses on the relevant information.

## Step 2: Generating the infrastructure report using AWS Trusted Advisor and AWS Config

**AWS Trusted Advisor service**
AWS Trust Advisor is used as input to generate health status report. It analyzes the AWS accounts and provides optimization recommendations following AWS best practices, including the AWS Well-Architected Framework. Below, in figure 5, is the view of the Trusted Advisor dashboard:

*Figure 5. AWS Trusted Advisor dashboard*

Each Trusted Advisor check includes a corresponding id, which can be used to query the details of the checks.

```
# Fetching all trusted advisor checks
import boto3

trusted_advisor_client = boto3.client("support")
trusted_advisor_checks = trusted_advisor_client.describe_trusted_advisor_checks(language='en')
```

The JSON payload contains details of the Trusted Advisor ID, name, description, and category, along with the metadata.

```
{
   "id":"Hs4Ma3G211",
   "name":"S3 buckets with versioning enabled should have lifecycle policies configured",
   "description":"Checks if Amazon Simple Storage Service (Amazon S3) version enabled buckets have li
   "category":"security",
   "metadata":[
      "Status",
      "Region",
      "Resource",
      "Last Updated Time"
   ]
}
```


Furthermore, the user can fetch the summary for each check.

```
support_boto_client.describe_trusted_advisor_check_summaries(checkIds=["Hs4Ma3G211"])
```

The response contains a flag if any resources have been highlighted for a given check, along with the number of resources flagged:


```
[
   {
      "checkId":"Hs4Ma3G211",
      "timestamp":"2023-10-03T07:47:50Z",
      "status":"warning",
      "hasFlaggedResources":true,
      "resourcesSummary":{
         "resourcesProcessed":8,
         "resourcesFlagged":3,
         "resourcesIgnored":0,
         "resourcesSuppressed":0
      },
      "categorySpecificSummary":{
         "costOptimizing":{
            "estimatedMonthlySavings":0.0,
            "estimatedPercentMonthlySavings":0.0
         }
      }
   }
```


These summarized and specific check overviews are then used as LLM inputs to form a health status overview of the account.

**AWS Config service**

The second service used for the health status report creation is AWS Config, which audits and evaluates compliance of the resources' configurations against organization's policies on a continual basis. The AWS Config dashboard is set forth in figure 6 below:

*Figure 6. AWS Config dashboard*

Generating the AWS Config report:

```
import boto3
from collections import Counter

config_client = boto3.client("config")

def get_number_of_resource_flagged_per_checks():
    """ Get count of flagged resources flagged for each check type"""
    rule_type = []
    resource = []
    resource_type = []
    compliance_rules = get_list_of_aws_config_checks()
    paginator = config_client\
        .get_paginator('get_compliance_details_by_config_rule')
    index = 1

    for rule in compliance_rules:
        response_iterator = paginator\
            .paginate(ConfigRuleName=rule,
                      ComplianceTypes=['NON_COMPLIANT'])
```

The AWS Config check report with the number of resources flagged by the check:

```
AWS Config checks :
————————————-
1.pcr-lambda-concurrency-check-prod-us-east-1-config-rule Resources flagged :4
2.securityhub-sagemaker-notebook-instance-inside-vpc-ad0fd80f Resources flagged :2
3.securityhub-subnet-auto-assign-public-ip-disabled-97cfdc6e Resources flagged :6
4.securityhub-s3-bucket-logging-enabled-903e9827 Resources flagged :12
5.securityhub-s3-account-level-public-access-blocks-periodic-379e7735 Resources flagged :1
6.securityhub-opensearch-data-node-fault-tolerance-4f307957 Resources flagged :1
7.securityhub-dynamodb-pitr-enabled-cbf1e710 Resources flagged :2
8.pcr-ec2-instance-profile-attached-prod-us-east-1-config-rule Resources flagged :2
9.securityhub-s3-event-notifications-enabled-0d95eb04 Resources flagged :12
10.securityhub-opensearch-logs-to-cloudwatch-5fae7804 Resources flagged :1
11.securityhub-dynamodb-autoscaling-enabled-efabed3e Resources flagged :2
12.securityhub-acm-certificate-expiration-check-b963b5e1 Resources flagged :3
13.securityhub-ec2-instance-no-public-ip-79ca01eb Resources flagged :1
14.securityhub-ec2-instance-managed-by-ssm-6032d57b Resources flagged :2
15.securityhub-ec2-security-group-attached-to-eni-periodic-2f6d8ffd Resources flagged :1
16.securityhub-ecr-private-image-scanning-enabled-aec982fc Resources flagged :1
17.securityhub-opensearch-audit-logging-enabled-bb6bebab Resources flagged :1
```

## Step 3: Building the conversational interface

Amazon Cognito, which empowers users to implement secure customer identity and access management (IAM), is used for the conversational interface. User requests are first passed through Amazon API Gateway, which is used to create, maintain,

and secure APIs at any scale. In turn, this invokes AWS Lambda functions, a serverless, event-driven compute service, that host the core functionality of the bot.

The ICCA has been designed to integrate seamlessly with multiple LLMs through REST APIs. This helps to ensure that the system is flexible enough to react and integrate quickly as new state-of-the-art models are developed in the rapidly evolving generative AI domain. The ICCA acts as a centralized interface, providing visibility into BMW's cloud infrastructure health. It assists by detecting issues, suggesting actions, and deploying changes on user request.

**A brief overview of LLM agents**

LLM agents are programs that use LLMs to decide when and how to use tools to complete complex tasks. With tools and task-planning abilities, LLM agents can interact with outside systems and overcome traditional limitations of LLMs, such as knowledge cutoffs, hallucinations, and imprecise calculations. Tools can take a variety of forms such as API calls, Python functions, or plug-ins.

So what does it mean for an LLM to pick tools and plan tasks? There are numerous approaches (such as ReAct, MRKL, Toolformer, HuggingGPT, and Transformer Agents) to using LLMs with tools, and advancements are happening rapidly. One simple way is to prompt an LLM with a list of tools and ask it (1) to determine if a tool is needed to satisfy the user query and if so, (2) to select the appropriate tool. Such a prompt is typically similar to the following example and may include a few short examples to improve the LLM's reliability in picking the right tool:

```
Your task is to select a tool to answer a user question.
You have access to the following tools.

search: search for an answer in FAQs
order: order items
noop: no tool is needed

{few short examples}

Question: {input}

Tool:
```

In figure 2, steps 4-10 depict the structure of the multiagent component, and the services connected to it. Different agents have access to a different set of tools to respond to the user query based on the conversation's current state. Amazon DynamoDB is used to store and share conversational history between different agents. All agents are maintained in a single Lambda function, and depending on the intent and state of the conversation, the appropriate agent is queried.

**Health Check Agent**

The Health Check agent summarizes account compliance: when a user sends a query and the intent identification component classifies it as Health Check intent, the query is re-rerouted to the Health Check agent. The agent has access to the previous conversation via the conversational memory stored in DynamoDB and can identify the account of interest and access it via AWS Trusted Advisor and AWS Config to create an infrastructure-wide report, see figure 7 below. In figure 8 below, the example report generated from the agent in the conversational interface is illustrated in a short video.

*Figure 7. Health Check agent workflow*

*Figure 8. Health Check agent chat demo*

**Issue Resolver Agent**

The Issue Resolver agent suggests changes to resolve issues that were identified in the Health Status report, describes recommendations in natural language, and generates code snippets following the workflow depicted below in figure 9.

*Figure 9. Issue Resolver agent workflow*

The recommendation code is generated in Terraform and Python using the Boto3 library. In figure 10 below, the example conversation with Issue Resolver agent is shown.

*Figure 10. Issue Resolver agent chat demo*

**Code and Pull Request Generator**

The Code and Pull Request Generator agent, depicted below in figure 11, is designed to resolve critical issues in the account from within the conversational interface. The agent is capable of incorporating metadata into the account while deploying the changes in the form of Terraform and Python boto3 using GitHub Actions.

*Figure 11. Code and Pull Request Generator agent workflow*

**Generic Chat Handler**

The Generic Chat Handler agent is designed to answer technical questions related to AWS services using the RAG pipeline built using Amazon Kendra. Figure 12 illustrates the Generic Chat Handler workflow, while figure 13 shows an example of generic query handling from the conversational interface.

*Figure 12. Generic Chat Handler workflow*

*Figure 13. Generic chat example*

## Conclusion

Generative AI is rapidly transforming how businesses can utilize cloud technologies. BMW's ICCA built using Amazon Bedrock demonstrates the immense potential of LLMs to enhance human capabilities. By providing developers expert guidance grounded in AWS best practices, this AI assistant enables BMW DevOps teams to review and optimize cloud architecture across thousands of AWS accounts.

BMW's innovative implementation of the ICCA using Amazon Bedrock highlights how organizations can use GenAI and industry-leading AI services from AWS to help drive innovation. With the ability to understand natural language, to reason, and to generate recommendations, GenAI can help provide immense value to enterprises on their cloud journey.

As this project shows, BMW is uniquely positioned at the intersection of the automotive and AI revolutions. The depth of experience that AWS has in AI and its proven enterprise capabilities make it the ideal resource for BMW to bring ambitious visions like the ICCA to life. Working alongside AWS, BMW is showcasing the art of the possible with LLMs in the present while also charting the course of future AI-powered cloud optimization.

To learn more about how AWS empowers organizations to design and build disruptive generative AI solutions visit AWS Generative AI Innovation Center.

TAGS: AWS Well-Architected, BMW

# Comments

Log in to comment